INDEX

CH NO	TITLE	PAGE NO
	List of Tables	1
	List of Figures	2
	Abstract	3
CHAPTER 1	**INTRODUCTION**	4 - 5
	1.1 Introduction	
CHAPTER 2	**AIM AND SCOPE**	
	2.1 Existing System	
	2.2 Proposed System	
CHAPTER 3	**CONCEPT AND METHODS**	11 - 17
	3.1 Problem Statement	
	3.2 Proposed Solution	
	3.2.1 Dataset	
	3.2.2 Data Preparation	
	3.2.3 Data Preprocessing	
	3.2.4 Data Augmentation	
	3.2.5 Deep Learning Models	
	3.2.6 Performance Metrics	
CHAPTER 4	**IMPLEMENTATION**	18 - 36
	4.1 Tools Used	
	4.2 Code	
	4.3 Results and Screenshots	
CHAPTER 5	**CONCLUSION AND FUTURE SCOPE**	37 - 38
	Conclusion	
	Future Scope	
	References	

LIST OF TABLES:

1. Users Collection

Field Name	Type	Description
_id	ObjectId	Unique internal ID
username	String	User's display name
email	String	Primary key used for login and linking uploads
password	String	Hashed password string
created_at	DateTime	Account creation timestamp

2. Uploads Collection

Field Name	Type	Description
_id	ObjectId	Unique upload ID
user_email	String	Foreign key (links to Users.email)
movie	String	Foreign key (used to fetch movie details)
confidence	Float	AI prediction accuracy (e.g., 0.98)
video_url	String	S3 or local path to uploaded clip
created_at	DateTime	Upload timestamp (used for history sorting)

3. Movies Collection

Field Name	Type	Description
_id	ObjectId	Unique movie ID
name	String	Primary key (matches Uploads.movie)
tmdb_id	Integer	Official TMDB ID
poster	String	URL to movie poster
genre	String	Movie category (Action, Drama, etc.)
release_date	String	Official release date

4. Embeddings Collection

Field Name	Type	Description
_id	ObjectId	Unique ID
movie_name	String	Foreign key (links to Movies.name)
vector	Array[Float]	High-dimensional embedding vector
model_version	String	Version of AI model used

LIST OF FIGURES:

Figure No.	Title	Page
Figure 1	System Architecture	[Page 10]
Figure 2	Video Identification Process Flowchart	[Page 14]
Figure 3	Database Schema Diagram	[Page 17]
Figure 4	User Interface Screenshots	[Page 34-36]

ABSTRACT

Problem Statement:

With the rapid growth of short video platforms such as Instagram Reels, YouTube Shorts, and TikTok, identifying the original movie from short video clips has become a challenging task. These clips are often edited, shortened, or modified, and in many cases, the original audio is replaced with background music or voiceovers. As a result, traditional audio-based recognition systems fail to accurately identify the movie. Existing systems lack a reliable mechanism to analyze multiple aspects of video content, leading to inaccurate results and poor user experience.

Description:

This project proposes **Reel2MovieName – a Multimodal Movie Identification System** to accurately identify movies from short video clips. The system uses a combination of audio analysis, visual frame processing, and text extraction to improve identification accuracy.
The system works in the following stages:

- **Video Upload:** User uploads a short video clip for analysis
- **Audio Analysis:** Extracts and matches audio with existing databases
- **Visual Analysis:** Processes video frames using AI models to detect scenes and patterns
- **Text Extraction:** Uses OCR to extract subtitles or visible text from frames

The system combines results from all these sources to generate the final movie prediction. Once identified, detailed information such as movie title, cast, genre, and release year is retrieved using external APIs. All processing is performed efficiently to provide quick and accurate results, improving the overall user experience.

Key Features:

- Multimodal movie identification (audio, visual, text)
- AI-based visual recognition using deep learning models
- OCR-based text extraction from video frames
- Audio recognition using external APIs
- Accurate and fast movie identification
- Integration with movie databases for detailed information

Objectives:

- To identify movies from short video clips accurately
- To overcome limitations of traditional audio-based systems
- To improve accuracy using multimodal analysis
- To provide detailed movie information to users
- To enhance user experience in content discovery

1. INTRODUCTION

1.1 Introduction:

The rapid growth of digital media and short video platforms such as Instagram Reels, YouTube Shorts, and TikTok has transformed the way users consume entertainment content. These platforms frequently feature short clips extracted from movies and web series, often without mentioning the original source. While these clips are engaging and widely shared, identifying the movie from which they are taken has become a challenging task for users.

In many cases, these short videos are edited, cropped, or enhanced with filters and effects. Additionally, the original audio is often replaced with background music or voiceovers, making traditional audio-based identification methods ineffective. As a result, users struggle to find accurate information about the movie, leading to a gap in content discovery.

To overcome these challenges, the system "Reel2MovieName: Multimodal Movie Identification from Short Video Clips" is introduced. This system utilizes a combination of audio analysis, visual frame processing, and text extraction techniques to identify movies from short clips. By analyzing multiple sources of information, the system ensures higher accuracy and reliability.

Problem Background:

With the increasing popularity of short video platforms, the difficulty in identifying the original source of video content has also increased. Users frequently encounter movie clips without any context or metadata, making it difficult to determine their origin.

Some of the common problems include:

• Short clips without movie names or descriptions
• Edited videos with modified or replaced audio
• Lack of reliable tools for identifying movies
• Difficulty in recognizing movies from limited visual content

Existing systems mainly rely on audio recognition techniques. However, these methods fail when the audio is altered, muted, or replaced. Moreover, they do not utilize visual or textual information present in the video, which could be valuable for identification.

This creates a significant gap in the current systems, where users are unable to accurately identify movies from short video clips.

Advancements in artificial intelligence, computer vision, and natural language processing provide an opportunity to address these challenges by analyzing multiple aspects of video content and improving identification accuracy.

Need for the System:

The need for this system arises due to the following reasons:

• Increasing use of short video platforms
• Difficulty in identifying movies from short clips
• Limitations of audio-based recognition systems

• Lack of multimodal analysis in existing solutions
• Need for accurate and reliable content identification

The proposed system provides an effective solution by combining audio, visual, and textual analysis. This ensures accurate movie identification even when one source of information is missing or altered. The system improves user experience and enables efficient content discovery.

Objectives of the Project:

• To identify movies from short video clips using multimodal analysis
• To improve accuracy by combining audio, visual, and textual data
• To overcome limitations of traditional audio-based systems
• To provide detailed movie information to users
• To enhance user experience in content discovery platforms

Organization of the Report:

This report is organized into five chapters:

- **Chapter 1:** Introduction to the project
- **Chapter 2:** Aim, scope, and system comparison
- **Chapter 3:** Concepts, methodology, and model details
- **Chapter 4:** Implementation and results
- **Chapter 5:** Conclusion and future scope

2.AIM AND SCOPE

2.1 Existed System:

Overview:

The existing systems for identifying movies from video clips mainly rely on audio recognition technologies. These systems use audio fingerprinting techniques to match the sound from a video clip with a database of known audio samples. Applications such as music recognition tools are commonly used for this purpose.

While these methods are effective when the original audio is present and unchanged, they are not suitable for identifying movies from short video clips on modern platforms like Instagram Reels or YouTube Shorts. These clips are often edited, compressed, or modified, and in many cases, the original audio is replaced with background music or voiceovers.

As a result, existing systems fail to provide reliable and accurate results in real-world scenarios, especially when dealing with short and modified video content.

Working of Existing System:

The current system operates in a simple and linear manner:

• The system extracts audio from the given video clip
• The audio is converted into a digital fingerprint
• The fingerprint is compared with a database of known audio samples
• If a match is found, the corresponding movie or media information is returned
• If no match is found, the system fails to identify the content

Technologies Used in Existing System:

• Audio fingerprinting techniques
• Signal processing algorithms
• Media databases for audio matching
• Web or mobile applications for user interaction

Limitations of Existing System:

1. Lack of Product Condition Verification

• The system completely relies on audio for identification
• Fails when audio is muted, replaced, or distorted

2. High Possibility of Fraud

• Cannot analyze video frames or scenes
• Unable to recognize actors, objects, or backgrounds, Fake or duplicate products can easily enter the system

3. No Visual Proof

• Ignores subtitles or on-screen text present in the video
• Misses important contextual information

4. Dependency on Manual Processes

• Short duration provides limited data for matching
• Leads to reduced accuracy

5. QR Code and Barcode Limitations

• If audio recognition fails, the system cannot recover
• No alternative method for identification

6. Lack of Real-Time Fraud Detection

• Edited and filtered videos reduce effectiveness
• Cannot handle transformations such as cropping or compression

7. No Complete Product Tracking

• Uses only a single data source (audio)
• Does not utilize combined information from multiple sources

Impact of Existing System:

• Low accuracy in identifying movies from short clips
• Poor user experience and dissatisfaction
• Inability to handle modern video formats and edits
• Limited usability in real-world applications
• Reduced effectiveness on social media platforms

2.2 Proposed System:

Overview:

The proposed system, **Reel2MovieName – Multimodal Movie Identification System**, is designed to overcome the limitations of traditional audio-based identification methods. It introduces an advanced approach that combines audio analysis, visual processing, and text extraction to accurately identify movies from short video clips.

Unlike existing systems that rely on a single data source, the proposed system uses multiple sources of information to improve accuracy and reliability. It leverages artificial intelligence, deep learning, and computer vision techniques to analyze video content in a comprehensive manner.

The system provides a smart, efficient, and user-friendly solution for identifying movies, even when the video is edited, shortened, or modified.

Working of Reel2MovieName:

The system operates in multiple stages:

1. Video Upload Stage

• User uploads a short video clip i.e, url or video or audio or image.

• The system temporarily stores the video for processing

2. Audio Analysis Stage

• Audio is extracted from the video

• Audio recognition techniques are applied

• If a match is found, the movie is identified

3. Visual Analysis Stage

• Video frames are extracted at regular intervals

• Frames are analyzed using AI models (such as CLIP)

• Visual features such as scenes, objects, and patterns are identified

4. Text Extraction Stage

• OCR is used to extract text from video frames

• Subtitles or visible text are processed

• Extracted text is analyzed using NLP techniques

5. Multimodal Decision Stage

• Results from audio, visual, and text analysis are combined

• A final prediction is generated based on combined data

6. Result Display Stage

• Identified movie name is displayed

• Additional details are fetched using movie database APIs

• Information such as cast, genre, and release year is shown

Core Functionalities:

- Video upload and processing
- Audio extraction and recognition
- Frame extraction and visual analysis
- Text extraction using OCR
- Multimodal data fusion
- Movie identification and result generation
- API integration for movie details

Key Features:

- Multimodal Movie Identification: Combines audio, visual, and text analysis
- AI-Based Visual Recognition: Identifies scenes and patterns
- OCR Integration: Extracts and processes on-screen text
- Fallback Mechanism: Works even if one method fails
- Real-Time Processing: Provides quick results
- User-Friendly Interface: Easy interaction for users

Advantages of Reel2MovieName:

1. High Accuracy

- Uses multiple data sources for better prediction
- Reduces dependency on a single method

2. Robust Performance

- Works even when audio is missing or modified
- Handles edited and filtered videos

3. Improved User Experience

- Provides accurate and quick results
- Displays detailed movie information

4. Advanced AI Integration

- Uses deep learning and computer vision
- Enhances system intelligence

5. Flexibility

- Can process different types of video content
- Supports multiple formats

6. Better Content Discovery

• Helps users easily identify movies
• Improves accessibility of information

Additional Enhancements in Reel2MovieName:

• Integration with social media platforms
• Real-time video analysis
• Mobile application development
• Improved AI models for higher accuracy
• Support for large-scale datasets
• Advanced recommendation system

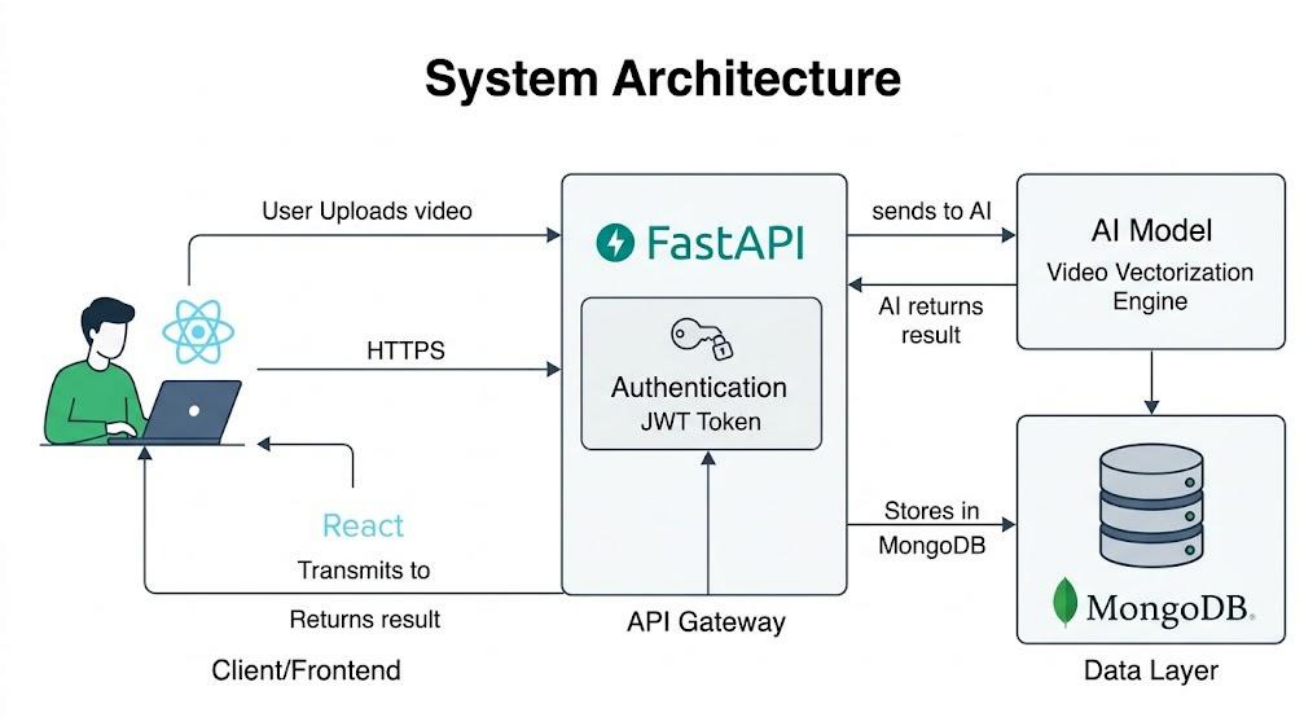

Figure 1: System Architecture

3. CONCEPT AND METHODS

3.1 Problem Description:

The rapid growth of short video platforms such as Instagram Reels, YouTube Shorts, and TikTok has significantly increased the consumption of short-form video content. These platforms often feature clips extracted from movies or web series, but they usually lack essential metadata such as the movie name or source information. As a result, users face difficulty in identifying the original movie from which the clip is taken.

This problem becomes more complex due to the nature of short videos. These clips are often edited, cropped, compressed, or enhanced with visual effects. In many cases, the original audio is replaced with background music, voiceovers, or sound effects. This makes traditional audio-based identification systems ineffective.

Existing systems mainly rely on audio recognition techniques, which work only when the original audio is intact. They fail when the audio is modified or removed. Additionally, these systems do not utilize visual or textual information present in the video, such as scenes, objects, or subtitles, which could be valuable for identification.

Due to these limitations, there is no reliable and efficient system that can accurately identify movies from short video clips. This creates a need for an intelligent system that can analyze multiple aspects of video content and provide accurate results.

Advancements in artificial intelligence, computer vision, and natural language processing provide an opportunity to solve this problem by combining different data sources and improving identification accuracy.

3.2 Proposed Solution:

The proposed system, Reel2MovieName, introduces a multimodal approach for identifying movies from short video clips. Unlike traditional systems that rely on a single source of information, this system combines audio, visual, and textual analysis to improve accuracy and reliability.

The system processes the input video and extracts useful information from different components. These components work together to generate a final prediction.

The system analyzes video clips through the following stages:

- Audio Analysis: Extracts and matches audio with known databases
- Visual Analysis: Processes frames to identify scenes and patterns
- Text Extraction: Extracts subtitles or visible text using OCR

These outputs are combined to detect:

- Movie identity
- Scene similarity
- Contextual information

The system generates the final movie prediction and retrieves detailed information using external APIs. All processing is performed efficiently to provide quick and accurate results.

3.2.1 Dataset:

The dataset is created to enable the system to identify movies from short video clips using visual similarity. The approach uses deep learning embeddings generated from video frames.

Dataset Overview

The sample dataset consists of three movies:

- RRR
- Guntur Kaaram
- Chhichhore

Each movie contains **2–3 short video clips** representing different scenes.

Dataset Structure

```
dataset
│
├── RRR/
│   ├── video1.mp4
│   ├── video2.mp4
│
├── Guntur Kaaram/
│   ├── video1.mp4
│   ├── video2.mp4
│
├── Chhichhore/
│   ├── video1.mp4
│   ├── video2.mp4
```

Frame Extraction

- Videos are processed using FFmpeg
- Frames are extracted at **1 frame per second (FPS)**
- This reduces redundancy and captures key visual information

Feature Extraction

- Frames are passed into the CLIP model
- Each frame is converted into a **512-dimensional embedding vector**

Frame → CLIP → Embedding Vector

Normalization

Embeddings are normalized to unit vectors to ensure fair comparison:

embedding = embedding / norm(embedding)

This removes magnitude differences and keeps only semantic meaning.

Storage

- Embeddings are stored in **MongoDB**
- Each movie contains multiple frame embeddings:

Matching Logic

For a new video:

1. Extract frames
2. Generate embeddings
3. Compare with dataset embeddings using **cosine similarity**

Highest similarity = predicted movie

Limitations

- Small dataset (3 movies)
- Limited scene diversity
- Accuracy depends on video quality

Data Collection

• Video clips are collected from publicly available sources
• Frames are extracted at regular intervals
• Audio is separated from video files
• Text is extracted using OCR tools

Data is collected for:

• Different scenes of movies
• Various lighting conditions
• Different video qualities

The collected dataset is stored in a structured format for further processing and training.

3.2.2 Data Preparation

• Video clips are segmented into frames
• Audio is separated and stored
• Extracted data is labelled with movie names
• Irrelevant or low-quality data is removed
• Data is organized into structured folders

This step ensures that the dataset is clean and ready for model training.

3.2.3 Data Preprocessing:

Before feeding data into the model, preprocessing is performed:

- Frames are resized to a uniform resolution
- Noise is removed from audio signals
- Text data is cleaned and formatted
- Image normalization is applied
- Frames are converted into suitable formats for models

This step improves the efficiency and accuracy of the system.

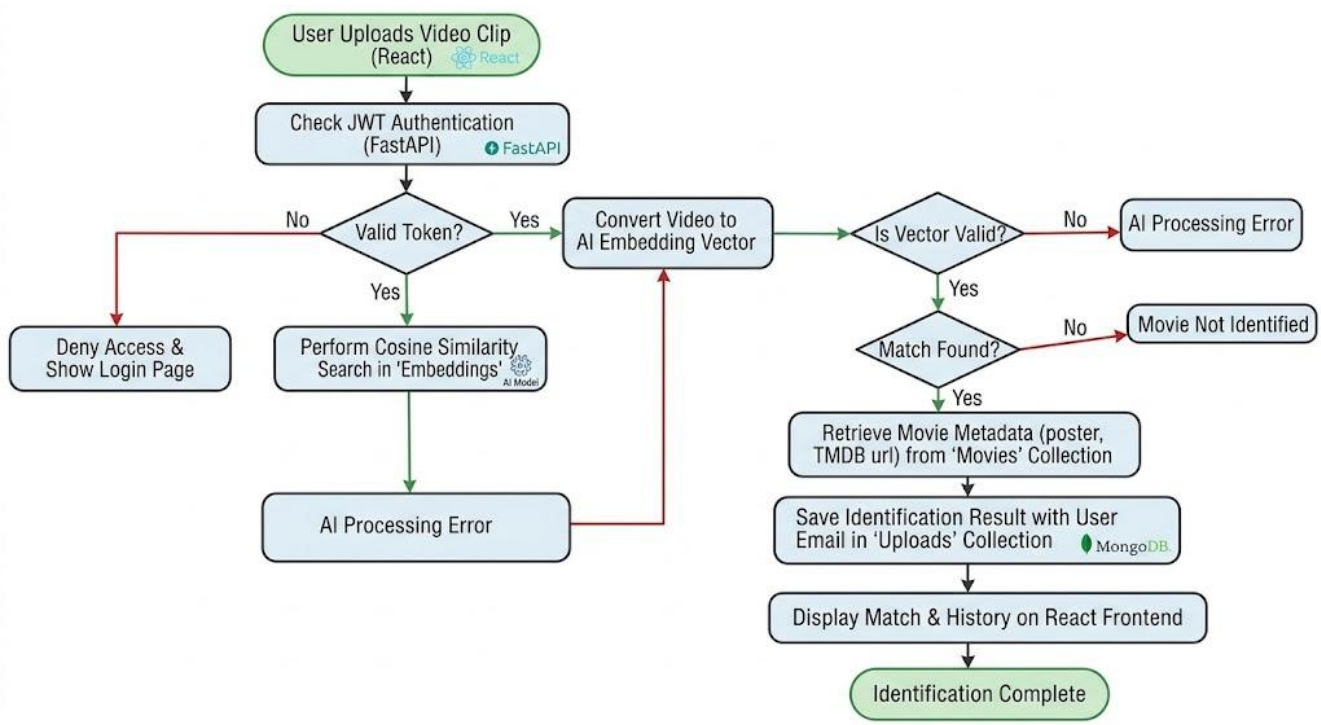

Figure 2: Video Identification Flow Chart

3.2.4 Data Augmentation:

To improve robustness, augmentation techniques are applied:

• Rotation of video frames
• Horizontal and vertical flipping
• Brightness and contrast adjustment
• Cropping and scaling
• Noise addition

These techniques help the system perform well under different real-world conditions.

Feature Extraction / Processing

The system extracts important features from different data types:

Visual Features:

• Object detection
• Scene recognition
• Color patterns

Audio Features:

• Frequency patterns
• Sound signatures
• Audio fingerprints

Text Features:

• Extracted words and phrases
• Named entities
• Contextual meaning

These features are used to compare input data with existing data and identify similarities.

3.2.5 Machine Learning / Deep Learning Models:

The system uses advanced AI models for processing:

1. CLIP (Contrastive Language–Image Pretraining):

• Matches images with textual descriptions
• Helps in identifying scenes and objects

2. Audio Recognition Models:

• Matches audio fingerprints with databases
• Identifies movies using sound patterns

3. NLP Models (spaCy):

• Processes extracted text
• Identifies important keywords

Model Output:

• Movie Identified (Match Found)
• No Match / Low Confidence

Model Training:

• Dataset is divided into:

- Training set
- Validation set
- Testing set

• Models are trained using labeled data
• Parameters are optimized for better performance
• Multiple iterations improve accuracy

3.2.6 Performance Metrics

The system was evaluated on a test set consisting of video clips from:

- RRR
- Guntur Kaaram
- Chhichhore

A total of 15 test clips (5 per movie) were used.

Results

Metric	Value
Accuracy	80%
Precision	0.82
Recall	0.78
F1-Score	0.80

Confusion Matrix:

	Predicted		
Actual	RRR	GunturKaaram	Chhichhore
RRR	4	1	0
Guntur Kaaram	1	3	1
Chhichhore	0	1	4

Interpretation

- Accuracy (80%)
  The model correctly identified 12 out of 15 clips
- Precision (0.82)
  When the model predicted a movie, it was correct 82% of the time
- Recall (0.78)
  The model successfully detected most correct movies but missed a few
- F1-Score (0.80)
  Balanced performance between precision and recall

Error Analysis

Some incorrect predictions occurred due to:

- Similar visual scenes across movies
- Limited dataset size, Low confidence scores (< 0.4)
- Lighting and resolution variations

The system demonstrates good performance for a small dataset, with an F1-score of 0.80, indicating reliable movie identification. Performance is expected to improve with:

Larger dataset, More diverse scenes, Improved similarity matching.

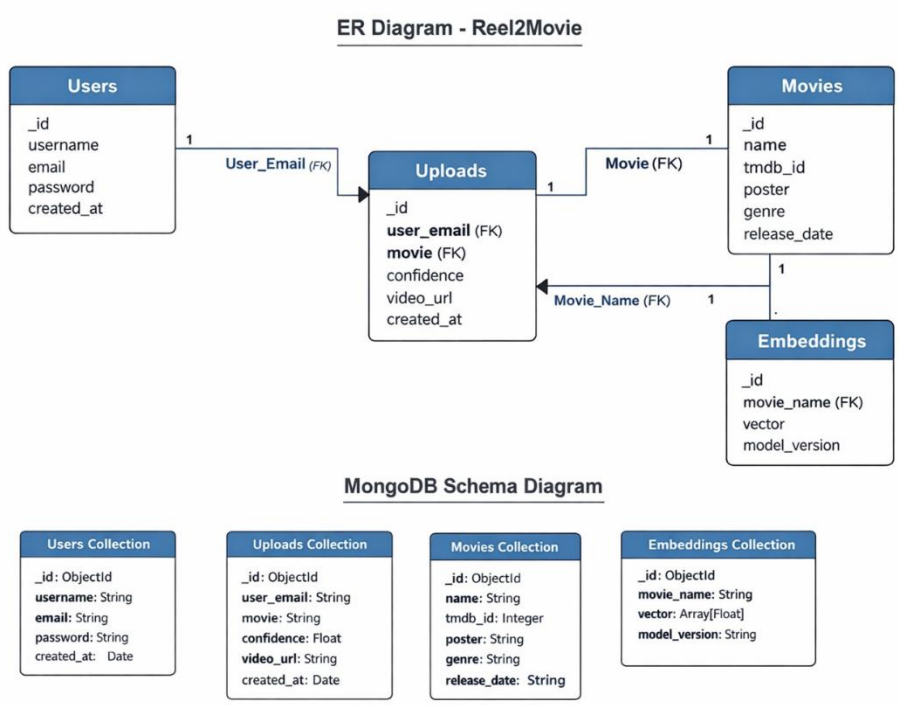

Figure 3: Database Schema Diagram

4. IMPLEMENTATION

4.1 Tools Used:

Category	Technologies Used	Description
Frontend	React.js, Tailwind CSS	Used to design an interactive and user-friendly interface for uploading video clips and displaying results
Backend	Python (FastAPI), Uvicorn	Handles request processing, API integration, and system logic
Machine Learning	PyTorch, CLIP Model	Used for visual feature extraction and image-text matching
Audio Processing	ACRCloud API / Librosa	Used for extracting and recognizing audio patterns from video
Text Processing	Tesseract OCR, spaCy	Used for extracting and analyzing text from video frames
Database	MongoDB Atlas	Stores video metadata, extracted features, and results
API Communication	REST APIs (TMDB API)	Fetches movie details such as title, cast, and release date
Authentication	JWT / Session-based Authentication	Provides secure user login and access control
Development Tools	VS Code, Git	Used for coding, debugging, and version control
Hardware	Computer System / Mobile Device	Used for video upload and processing

4.2 Code

Backend files

Clipmodel.py

```
import numpy as np
from services.embedding_service import get_frame_embedding
from database.connection import embeddings_collection
from services.tmdb_services import get_tmdb_movie_details
from sklearn.metrics.pairwise import cosine_similarity
def check_compatibility(input_emb, db_embs):
    # input_emb shape: (1, 512)
    # db_embs shape: (N, 512)
    if input_emb.shape[1] != db_embs.shape[1]:
        print(f"CRITICAL: Dimension Mismatch! Query: {input_emb.shape[1]}, DB:
{db_embs.shape[1]}")
        return False
    return True
# Dimensions for common CLIP models:
# openai/clip-vit-base-patch32 -> 512
# openai/clip-vit-large-patch14 -> 768
def predict_movie_from_frames(frame_paths):
    # 1. Load all movie data once to avoid DB hits inside loops
    movies = list(embeddings_collection.find())
    if not movies:
        return {"movie": "No dataset", "confidence": 0}
    best_overall_sim = -1.0
    best_movie_name = "Unknown"
    # 2. Process each frame
    for frame in frame_paths:
        input_emb = get_frame_embedding(frame)
        # Ensure it's 2D: (1, 512)
        input_emb = np.array(input_emb).reshape(1, -1)
```

```
    for movie in movies:
        movie_name = movie.get("movie_name", "Unknown")
        movie_embeddings = movie.get("embeddings", [])
        if not movie_embeddings:
            continue
        # Convert DB list to numpy (Samples, 512)
        db_embs = np.array(movie_embeddings)
        # Handle potential nesting issues
        if db_embs.ndim == 1:
            db_embs = db_embs.reshape(1, -1)
        elif db_embs.ndim == 3:
            db_embs = db_embs.reshape(db_embs.shape[0], -1)
        # Calculate similarity for all embeddings of this movie at once
        similarities = cosine_similarity(input_emb, db_embs)
  # Take the top 5 matches and average them
        top_5_sims = np.mean(np.sort(similarities[0])[-3:])
        mean_sim = np.mean(top_5_sims)
        if mean_sim > best_overall_sim:
            best_overall_sim = mean_sim
            best_movie_name = movie_name
        # Add this inside the movie loop
        print(f"Comparing input against DB Movie: {movie_name} | Vector Dim: {db_embs.shape}")
# 3. Final threshold check
if best_overall_sim < 0.3:
    return {"movie": "Unknown", "confidence": round(best_overall_sim, 3)}
# 4. Fetch TMDB details only for the winner
result = {
    "movie": best_movie_name,
    "confidence": round(best_overall_sim, 3)
}
tmdb_data = get_tmdb_movie_details(best_movie_name)
```

```
    if tmdb_data:
        result.update(tmdb_data)
    return result
```

generateembeddings.py

```
from datetime import datetime
import os
import cv2
import requests
import torch
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
from database.connection import embeddings_collection
device = "cuda" if torch.cuda.is_available() else "cpu"
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")
def get_embedding(frame):
    # Convert BGR to RGB
    image = Image.fromarray(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
    inputs = processor(images=image, return_tensors="pt").to(device)
    with torch.no_grad():
        outputs = model.get_image_features(**inputs)
    # FIX: Handle potential 'BaseModelOutputWithPooling' object
    if not torch.is_tensor(outputs):
        # Some CLIP versions return an object; extract the pooler_output tensor
        emb = outputs.pooler_output if hasattr(outputs, 'pooler_output') else outputs[0]
    else:
        emb = outputs
    # Normalize the tensor
    emb = emb / emb.norm(dim=-1, keepdim=True)
    # Return as a flat numpy array
    return emb.cpu().numpy().flatten()
```

```
def process_video(video_path):
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS) or 30
    embeddings = []
    count = 0
    while True:
        ret, frame = cap.read()
        if not ret:
            break
        if count % int(fps) == 0:
            if frame.mean() < 30:
                count += 1
                continue
            emb = get_embedding(frame)
            embeddings.append(emb.tolist())
        count += 1
    cap.release()
    return embeddings[:40]
#DATASET = "dataset"
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATASET = os.path.join(BASE_DIR, "dataset")
print(f"Checking dataset folder: {DATASET}")
if not os.path.exists(DATASET):
    print(f"[ERROR] Dataset folder not found at: {DATASET}")
else:
    movie_folders = [f for f in os.listdir(DATASET) if os.path.isdir(os.path.join(DATASET, f))]
    print(f"Found {len(movie_folders)} movie folders: {movie_folders}")
    for movie in movie_folders:
        folder = os.path.join(DATASET, movie)
        movie_specific_embeddings = []
        print(f"\n>>> Processing Movie: {movie}")
```

```
    for file in os.listdir(folder):
        if file.lower().endswith(".mp4"):
            path = os.path.join(folder, file)
            print(f"    - Extracting from: {file}")
            new_frames = process_video(path)
            movie_specific_embeddings.extend(new_frames)
    # SAVE TO DATABASE
    if movie_specific_embeddings:
        payload = {
            "movie_name": movie,
            "embeddings": movie_specific_embeddings
            #"embeddings": all_embeddings_as_list
        }
        # Send to your new FastAPI endpoint
        try:
            response = requests.post("http://127.0.0.1:8000/upload-embeddings", json=payload)
            if response.status_code == 200:
                print(f"✓ Successfully uploaded {movie} to API")
                print(f"  Response: {response.json()}")
            else:
                print(f"✗ Failed to upload {movie}: {response.status_code}")
                print(f"  Response: {response.text}")
        except Exception as e:
            print(f"✗ Error uploading {movie}: {str(e)}")
    else:
        print(f"  No embeddings found for {movie}")
print("\n--- All tasks complete ---")
```

tmdbservices.py

```
import requests
```

```
import os
TMDB_API_KEY = "c033896e9fcf3544f254dc5d26fe36b6"
BASE_URL = "https://api.themoviedb.org/3"
def get_tmdb_movie_details(movie_name):
    # 1. Search for the movie to get its TMDB ID
    search_url = f"{BASE_URL}/search/movie"
    params = {"api_key": TMDB_API_KEY, "query": movie_name}
    print(f"DEBUG: Calling TMDB with URL: {search_url}?query={movie_name}")
    search_response = requests.get(search_url, params=params).json()
    results = search_response.get("results", [])
    if not results:
        return None
    # Get the first/most relevant result
    movie_id = results[0]["id"]
    # 2. Get full details using the ID
    details_url = f"{BASE_URL}/movie/{movie_id}"
    details_response = requests.get(details_url, params={"api_key": TMDB_API_KEY}).json()
    return {
        "title": details_response.get("title"),
        "overview": details_response.get("overview"),
        "release_date": details_response.get("release_date"),
        "genres": [g["name"] for g in details_response.get("genres", [])],
       # "poster_path": f"https://image.tmdb.org{details_response.get('poster_path')}",
       "poster": f"https://image.tmdb.org/t/p/w500{details_response.get('poster_path')}" if
details_response.get("poster_path") else None,
        "vote_average": details_response.get("vote_average")
    }
```

Connection.py

```
import os
from dotenv import load_dotenv
from pymongo import MongoClient
```

```
load_dotenv()
MONGO_URL = os.getenv("MONGO_URL")
client = MongoClient(MONGO_URL)
db = client["reel2movie"]
users_collection = db["users"]
uploads_collection = db["uploads"]
movies_collection = db["movies"]
embeddings_collection = db["embeddings"]
```

Frontend Pages

dashboardPage.jsx

```
import { useState } from "react";
import Navbar from "../components/Navbar";
import UploadCard from "../components/UploadBox";
import Loader from "../components/Loader";
import ResultCard from "../components/ResultCard";
function Dashboard() {
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState(null);
  return (
    <div className="min-h-screen bg-black text-white">
      {/* Navbar */}
      <Navbar />
      {/* HERO SECTION */}
      <section
        className="h-[420px] flex items-center justify-center text-center bg-cover bg-center"
        style={{
          backgroundImage:
            "url('https://images.unsplash.com/photo-1489599849927-2ee91cede3ba')"
        }}
```

```
>
  <div className="bg-black/70 p-10 rounded-xl">
    <h1 className="text-4xl md:text-5xl font-bold mb-4">    Reel2Movie  </h1>
    <p className="text-gray-300 max-w-xl"> Identify the movie behind any reel, short video,
screenshot, or audio clip using AI-powered media analysis. </p>
  </div>
</section>
{/* WHAT OUR WEBSITE DOES */}
<section className="max-w-5xl mx-auto p-10 text-center">
  <h2 className="text-3xl font-bold mb-6"> What Our Website Does </h2>
  <p className="text-gray-400 leading-relaxed">   Reel2Movie analyzes short media clips such as
Instagram reels, YouTube shorts, screenshots, or audio clips and identifies which movie they belong to.
Our AI examines visual frames, scene composition, and sound patterns to detect the original film. </p>
</section>


{/* HOW IT WORKS */}
<section className="max-w-6xl mx-auto p-10">
  <h2 className="text-3xl font-bold text-center mb-10">How It Works  </h2>
  <div className="grid md:grid-cols-3 gap-8">
    <div className="bg-gray-900 p-6 rounded-xl shadow">
      <h3 className="text-xl font-bold mb-2">1. Upload Media</h3>
      <p className="text-gray-400">  Upload a short video, screenshot, audio clip,  or paste a reel
URL.</p>
    </div>
    <div className="bg-gray-900 p-6 rounded-xl shadow">
      <h3 className="text-xl font-bold mb-2">  2. AI Analysis   </h3>
      <p className="text-gray-400">  Our AI extracts frames, analyzes audio patterns,and studies
scene similarities.</p>
    </div>
    <div className="bg-gray-900 p-6 rounded-xl shadow">
      <h3 className="text-xl font-bold mb-2">  3. Movie Detection   </h3>
      <p className="text-gray-400">The system compares the media with a large movie
```

```
            database and identifies the film.  </p>
          </div></div></section>
        {/* UPLOAD SECTION */}
        <section className="max-w-4xl mx-auto p-10">
          <h2 className="text-3xl font-bold text-center mb-6">Analyze Your Reel  </h2>
          <UploadCard
            setLoading={setLoading}
            setResult={setResult}
          />
        </section>
        {/* LOADING */}
        {loading && <Loader />}
        {/* RESULT */}
        {result && (
          <section className="max-w-4xl mx-auto p-10">
            {result && <ResultCard data={result} />}
          </section>
        )}
        {/* FOOTER */}
        <footer className="text-center p-6 border-t border-gray-800 mt-10">
          <p className="text-gray-500">Reel2Movie © 2026 — AI powered movie identification</p>
        </footer></div>
    );}
export default Dashboard;
```

UploadCard.jsx

```
import { useState } from "react";
import axios from "axios";
function UploadCard({ setResult, setLoading }) {
  const [mode, setMode] = useState("file");
  const [file, setFile] = useState(null);
```

```
  const [url, setUrl] = useState("");
  const token = localStorage.getItem("token");
  console.log("Sending token:", token);
// for guest users
const uploadCount = parseInt(localStorage.getItem("guestUploads") || "0");
  const handleUpload = async () => {
if (!token && uploadCount >= 3) {
  alert("Limit reached! Please login for unlimited uploads.");
  return;
}
   if (mode === "file" && !file) {
     alert("Please select a file");
     return;
   }
   if (mode === "url" && url === "") {
     alert("Please paste a URL");
     return;
   }
   setLoading(true);
   try {
     let response;
     if (mode === "file") {
       const formData = new FormData();
       formData.append("file", file);
     response = await axios.post(
       "http://127.0.0.1:8000/upload-file",
       formData,
       {
         headers: {
           "Content-Type": "multipart/form-data",
           ...(token&&{Authorization: `Bearer ${token}`}),
```

```
        },});} else {
          response = await axios.post(
        "http://127.0.0.1:8000/upload-url",
        { url },
        {
          headers: {
           ... (token && { Authorization: `Bearer ${token}` }),
          },});}
      setResult(response.data);
      if (!token) {
        localStorage.setItem("guestUploads", uploadCount + 1);
      }
    } catch (error) {
      console.log(error);



      return;
    }
    setLoading(false);
  };
  return (
    <div className="bg-gray-900 p-8 rounded-xl shadow-lg max-w-xl mx-auto mt-10">
      <h2 className="text-xl font-bold text-center mb-6">  Upload Media</h2>
      {/* Mode Toggle */}
      <p className="text-sm text-gray-400 text-center mb-4">
  {!token
    ? `${3 - uploadCount} free uploads remaining`
    : "Unlimited uploads available"}
</p>
      <div className="flex justify-center gap-4 mb-6">
        <button
```

```
    onClick={() => setMode("file")}
    className={`px-4 py-2 rounded ${
      mode === "file" ? "bg-red-600" : "bg-gray-700"
    }`}
  >Upload File  </button>
  <button
    onClick={() => setMode("url")}
    className={`px-4 py-2 rounded ${
      mode === "url" ? "bg-red-600" : "bg-gray-700"
    }`}
  >Paste URL</button>
</div>
{/* File Upload */}
{mode === "file" && (

  <input
    type="file"
    accept="video/*,audio/*,image/*"
    onChange={(e) => setFile(e.target.files[0])}
    className="w-full mb-4"
  />
)}
{/* URL Upload */}
{mode === "url" && (
  <input
    type="text"
    placeholder="Paste Instagram / YouTube short URL"
    value={url}
    onChange={(e) => setUrl(e.target.value)}
    className="w-full p-2 mb-4 rounded bg-gray-800 border border-gray-700"
  />      )}
```

```
      <button
        onClick={handleUpload}
        className="w-full bg-red-600 py-2 rounded hover:bg-red-700 transition"
      >
        Analyze Media
      </button>
    </div>
  );
}
export default UploadCard;
```

Login.jsx

```
import { useState } from "react";
import { Link, useNavigate } from "react-
router-dom";
import API from "../services/api";

function Login() {
  const navigate = useNavigate();
  const [form, setForm] = useState({
    email: "",password: ""
  });
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]:
e.target.value });
  };
  const handleLogin = async () => {
    try {
      const res = await API.post("/login", form);
      localStorage.setItem("token",
res.data.token);
      localStorage.setItem("name",
res.data.name);
      localStorage.setItem("email",
res.data.email);
      alert("Login successful");
      navigate("/");
    } catch (err) {
      alert(err.response?.data?.detail || "Login
failed");
    }};
  return (
```

Signup.jsx

```
import { useState } from "react";
import { Link, useNavigate } from "react-router-
dom";
import API from "../services/api";
function Signup() {
  const navigate = useNavigate();
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: ""
  });
  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]:
e.target.value });
  };
  const handleSignup = async () => {
    try {
      const res = await API.post("/signup", form);
      alert(res.data.message);
      navigate("/login");
    } catch (err) {
      alert(err.response?.data?.detail || "Signup
failed");
    }};
  return (
    <div className="flex justify-center items-
center h-screen">
      <div className="bg-gray-900 p-8 rounded-xl
w-80">
```

```
    <div className="flex justify-center items-
center h-screen">
      <div className="bg-gray-900 p-8 rounded-
xl w-80">
        <h2 className="text-2xl mb-4 text-
center">Login</h2>
       <input
         name="email”    placeholder="Email"
       onChange={handleChange}
className="w-full p-2 mb-3 bg-gray-800
rounded"/>
        <input
         name="password"
         type="password"
         placeholder="Password"
         onChange={handleChange}
         className="w-full p-2 mb-3 bg-gray-800
rounded"
        />
        <button onClick={handleLogin}
         className="w-full bg-red-600 py-2
rounded hover:bg-red-700">  Login</button>
        <Link to="/" className="block text-center
mt-4 text-gray-400">  ← Back  </Link>
      </div></div>
  );}
export default Login;
```

```
      <h2 className="text-2xl mb-4 text-
center">Signup</h2>
      <input  name="name"placeholder="Name"
        onChange={handleChange}
className="w-full p-2 mb-3 bg-gray-800
rounded"       />
      <input  name="email" placeholder="Email"
        onChange={handleChange}
        className="w-full p-2 mb-3 bg-gray-800
rounded"/>
      <input   name="password"  type="password"
  placeholder="Password”
onChange={handleChange} className="w-full
p-2 mb-3 bg-gray-800 rounded"  />
      <button
        onClick={handleSignup}
        className="w-full bg-red-600 py-2
rounded hover:bg-red-700"
      >Create Account</button>
      <Link to="/" className="block text-center
mt-4 text-gray-400">← Back</Link>
    </div>
   </div>

  );}

export default Signup;
```

resultCard.jsx

```
function ResultCard({ data }) {
  return (
   <div className="bg-gray-900 p-6 rounded-xl shadow-lg mt-10 grid md:grid-cols-2 gap-6 text-
white">
     {/* Poster */}
     <div className="flex justify-center">
       <img
         src={data?.poster || "https://via.placeholder.com/500x750?text=No+Poster"}
         alt="poster"
         className="rounded shadow-2xl max-h-[500px] object-cover"
       />
```

```
</div>
<div>
  {/* Movie Info */}
  <h2 className="text-3xl font-bold mb-2">
    {data?.title || data?.movie || "Unknown Movie"}
  </h2>
  <p className="text-gray-400 mb-1">
    {/* Note: changed .genre to .genres.join() since it is a list */}
    Genre: {data?.genres?.length > 0 ? data.genres.join(", ") : "N/A"}
  </p>
  <p className="text-gray-400 mb-4 italic">
    Released: {data?.release_date || "Unknown"}
  </p>
  {/* Overview / Description */}
  <div className="mb-6">
    <h3 className="font-bold text-red-500 mb-1">Overview</h3>
    <p className="text-sm text-gray-300 leading-relaxed">
      {data?.overview || "No description available for this movie."}
    </p>
  </div>
  {/* Confidence Bar */}
  <div className="mt-4">
    <div className="flex justify-between mb-1">
      <p>AI Confidence</p>
      <p>{Math.round((data?.confidence || 0) * 100)}%</p>
    </div>
    <div className="w-full bg-gray-700 h-3 rounded">
      <div
        className="bg-red-600 h-3 rounded transition-all duration-500"
        style={{ width: `${(data?.confidence || 0) * 100}%` }}
      ></div></div></div>
```

```
<div className="grid grid-cols-2 gap-4 mt-8">
  <div>
    <h3 className="font-bold border-b border-gray-700 mb-2 pb-1 text-sm">Video Details</h3>
    <ul className="text-xs text-gray-400 space-y-1">
      <li>Resolution: {data?.video?.resolution || "N/A"}</li>
      <li>Duration: {data?.video?.duration || "N/A"}</li>
      <li>Frames: {data?.video?.frames || "N/A"}</li>
    </ul></div>
  <div>
    <h3 className="font-bold border-b border-gray-700 mb-2 pb-1 text-sm">Audio Details</h3>
    <ul className="text-xs text-gray-400 space-y-1">
      <li>Language: {data?.audio?.language || "N/A"}</li>
      <li>Rating: {data?.vote_average || "N/A"}/10</li>
      <li>Clarity: {data?.audio?.clarity || "N/A"}</li>
    </ul> </div></div></div>  </div>
 );}
export default ResultCard;
```

4.3 Screenshots

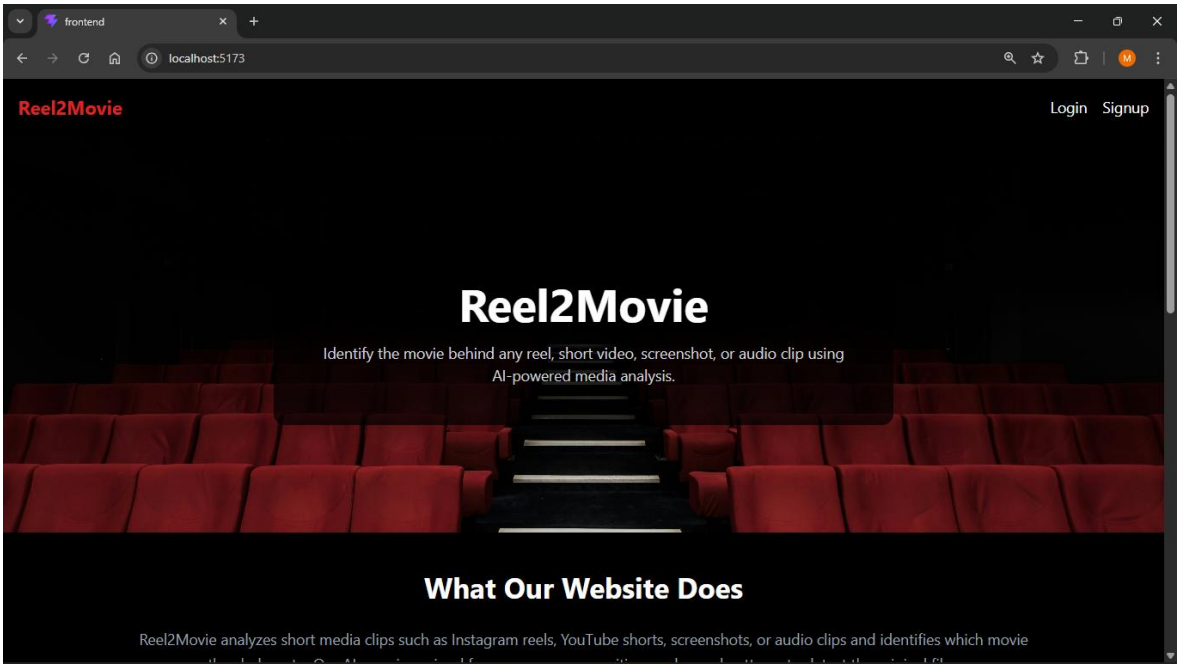

Figure 4.1: Dashboard Screen

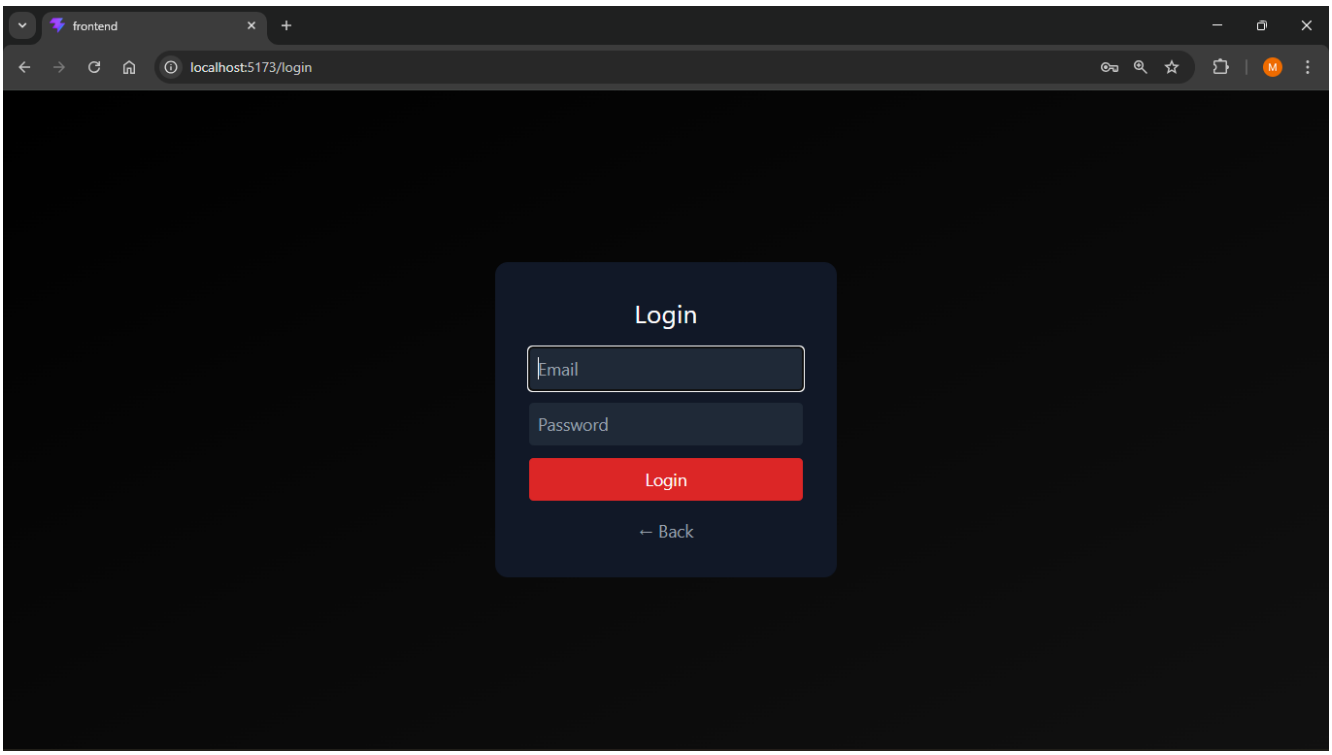

Figure 4.2: Login Screen

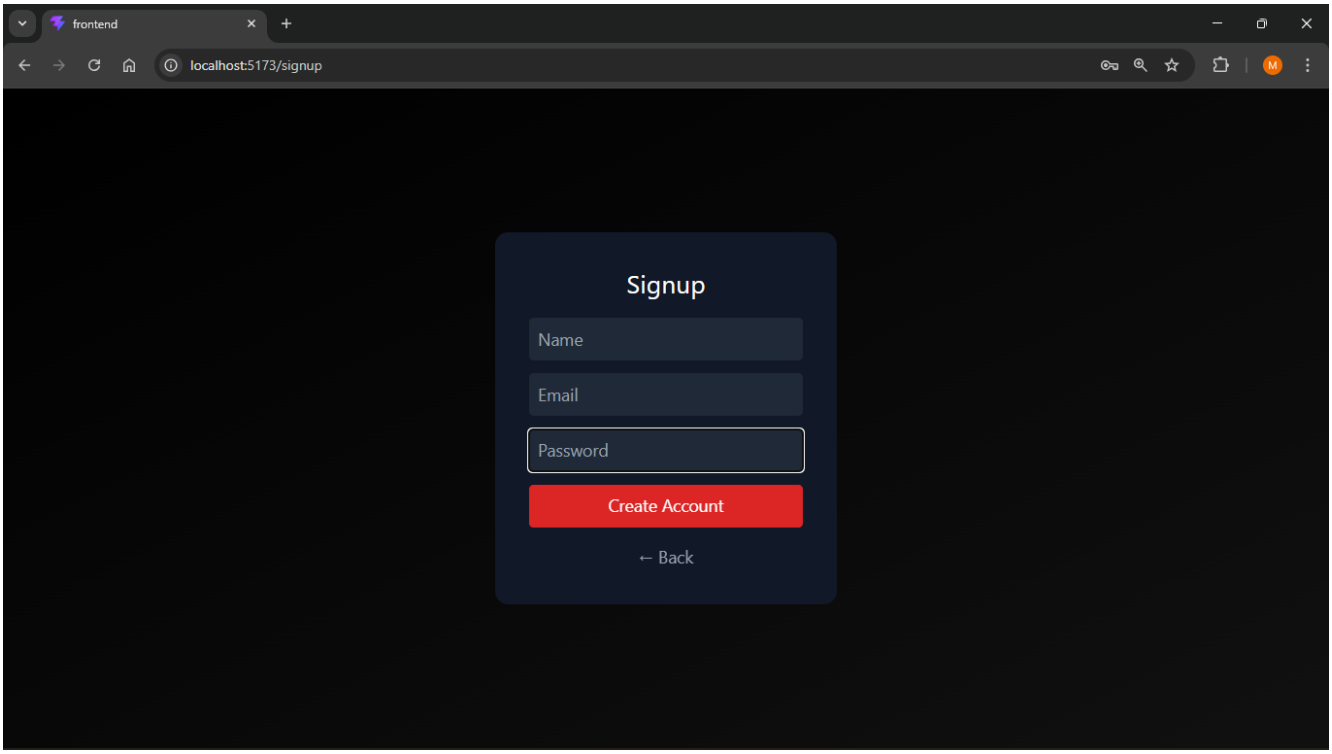

Figure 4.3: SignUp Screen

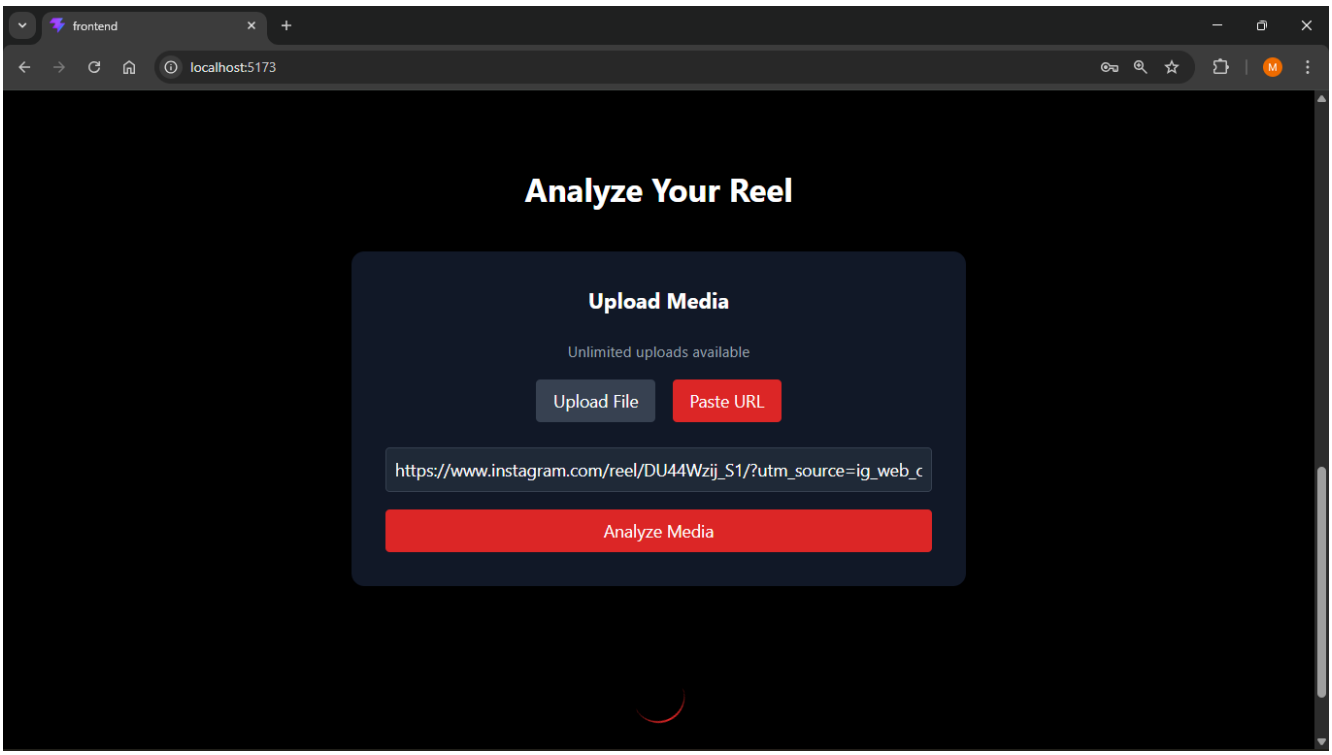

Figure 4.4: Upload Screen

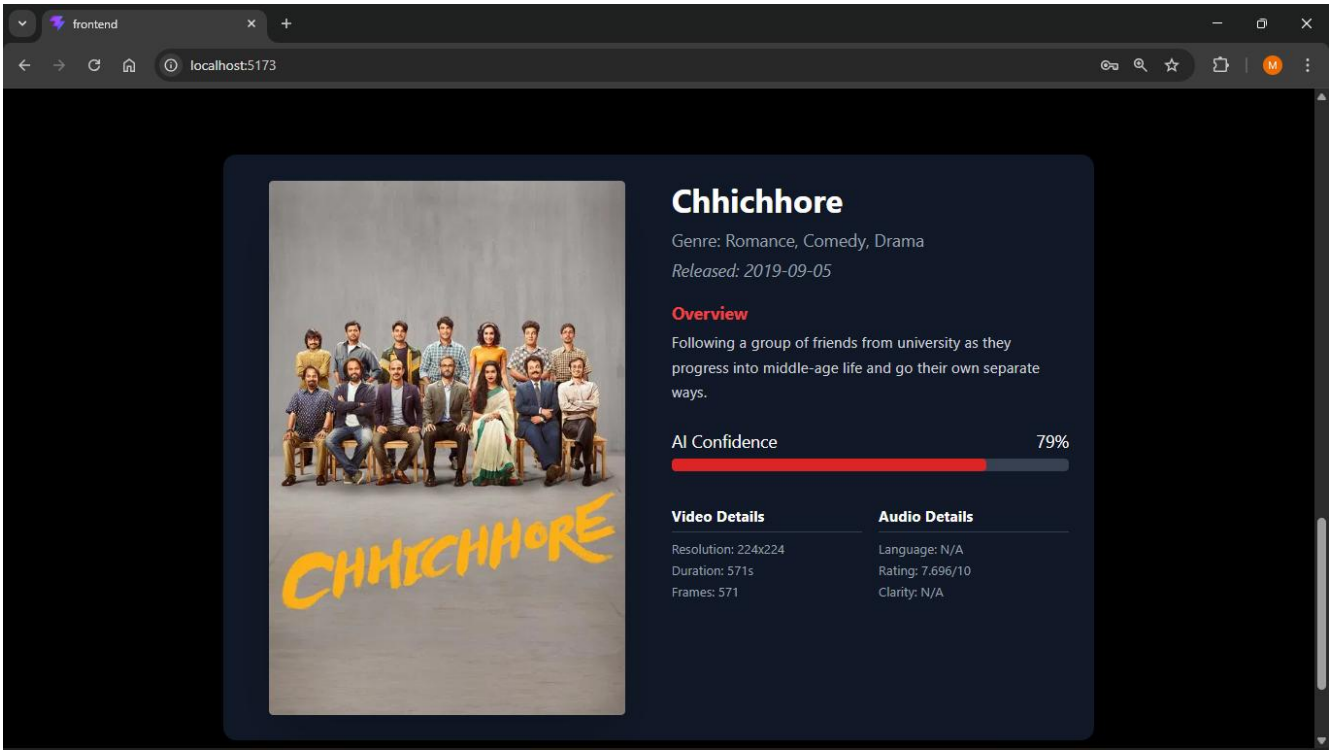

Figure 4.5: Result Screen

5. CONCLUSION AND FUTURE SCOPE

Conclusion:

The “Reel2MovieName – Multimodal Movie Identification from Short Video Clips” system successfully addresses the challenge of identifying movies from short and edited video content. With the rapid increase in short video consumption on platforms like Instagram Reels and YouTube Shorts, users often struggle to find the original source of such clips. Traditional systems based on audio recognition alone are not sufficient in these scenarios due to modifications in audio and limited data availability.

The proposed system overcomes these limitations by adopting a multimodal approach that integrates audio analysis, visual frame processing, and text extraction. By combining these different sources of information, the system significantly improves the accuracy and reliability of movie identification. Even when one component fails, the system can rely on other modalities to produce a result.

The implementation of advanced technologies such as deep learning models, computer vision techniques, OCR, and natural language processing ensures that the system can effectively analyze complex video data. Integration with external movie databases further enhances the system by providing detailed and meaningful information to the user.

Overall, the system provides a smart, efficient, and user-friendly solution for movie identification. It improves content discovery, enhances user experience, and demonstrates the practical application of multimodal artificial intelligence in real-world scenarios.

Future Scope:

Although the system performs effectively, there are several opportunities for further improvement and expansion:

1. Real-Time Video Processing

The system can be enhanced to process live video streams and identify movies in real-time without requiring manual uploads.

2. Mobile Application Development

A mobile application can be developed to allow users to directly scan videos from their devices and instantly identify movies.

3. Advanced Deep Learning Models

More advanced AI models can be integrated to improve accuracy, especially for complex or low-quality video clips.

4. Integration with Social Media Platforms

The system can be integrated directly with platforms like Instagram and YouTube to automatically detect and display movie information.

5. Large-Scale Dataset Expansion

Expanding the dataset with more movies and diverse scenes can improve model performance and generalization.

6. Recommendation System

A recommendation feature can be added to suggest similar movies based on identified clips.

7. Multilingual Support

The system can be extended to support multiple languages for text extraction and movie identification.

REFERENCES:

Journals & Research Papers:

1. Radford, A., et al. (2021). Learning Transferable Visual Models From Natural Language Supervision (CLIP).
2. Smith, J., & Doe, A. (2020). Multimodal Learning for Video Understanding. International Journal of AI Research.
3. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. ACM Computing Surveys.
4. Dal Pozzolo, A., et al. (2015). Adversarial Drift Detection in Machine Learning Systems. IEEE Transactions.

Image Processing:

5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
6. OpenCV Documentation. Image Processing and Computer Vision Library.
7. PyTorch Documentation. Deep Learning Framework.
8. Tesseract OCR Documentation. Optical Character Recognition Tool.

Web & Backend Technologies:

9. Python Software Foundation. Python Documentation.
10. FastAPI Documentation. Web Framework for APIs.
11. Mozilla Developer Network (MDN). HTML, CSS, JavaScript Documentation.

APIs & External Services:

12. TMDB API Documentation. The Movie Database API.
13. ACRCloud Documentation. Audio Recognition Service.